Knowledge and Data

Working Group

Week 3: Triple Stores & RDFS

Question 1—Triple Stores

Many (relational) databases and triple stores make use of a *dictionary encoding*.

Explain, in your own words, the concept of a dictionary encoding, and why it makes sense to use one in a triple store.

Question 1—Triple Stores

Many (relational) databases and triple stores make use of a *dictionary encoding*.

Explain, in your own words, the concept of a dictionary encoding, and why it makes sense to use one in a triple store.

Definition

A dictionary encoding D defines a one-to-one mapping between keys k and pointers p

Question 1—Triple Stores

Many (relational) databases and triple stores make use of a *dictionary encoding*.

Explain, in your own words, the concept of a dictionary encoding, and why it makes sense to use one in a triple store.

Definition

A dictionary encoding D defines a one-to-one mapping between keys k and pointers p

In a triple store, dictionary encodings are a sensible idea because

- long URIs/IRIs are mapped to much shorter pointers (addresses),
- resulting in less memory use.

Question 2—Querying a Triple Store

Assume that there exists a triple store S that is accessible at:

`https://vu.example.org:5852/KandD/`

Task: Describe how you can get structured information from S.

Question 2—Querying a Triple Store

Assume that there exists a triple store S that is accessible at:

`https://vu.example.org:5852/KandD/`

Task: Describe how you can get structured information from S.

- Triple store S listens at port 5852 for `HTTP GET` requests.
- SPARQL queries can piggyback on a `HTTP GET` request to S.
- Such `HTTP` requests can be made by a browser, programming environment, or tool (e.g. cURL).
- Triple store S interprets the request, and executes the query.
- Triple store S responds with the results in the requested format.
- The results are printed raw, or are rendered in a browser or programming environment.

Question 2—Querying a Triple Store

Example:

```
1 curl -H "Accept:text/turtle" \
2       -G https://dbpedia.org/sparql \
3       --data-urlencode query="DESCRIBE <http://dbpedia.org/resource/Amsterdam>"
```

Output:

```
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix dbr: <http://dbpedia.org/resource/> .

dbr:Barend_Bonneveld dbo:deathPlace dbr:Amsterdam .
dbr:Barend_Joseph_Stokvis dbo:deathPlace dbr:Amsterdam ;
                          dbo:birthPlace dbr:Amsterdam .
dbr:Barent_Fabritius dbo:deathPlace dbr:Amsterdam .
dbr:Barlaeus_Gymnasium dbo:location dbr:Amsterdam .
...
```

Question 3—RDFS Inferencing

Assume the following inference rule:

Rule 9

if $(e, \texttt{rdf:type}, C_1) \wedge (C_1, \texttt{rdfs:subClassOf}, C_2)$
$\rightarrow (e, \texttt{rdf:type}, C_2)$

Let G be a knowledge graph, with triples:

(ex:Victor,	rdf:type,	ex:Teacher)
(ex:Teacher,	rdfs:subClassOf,	ex:EducationalStaff)
(ex:EducationalStaff,	rdfs:subClassOf,	foaf:Person)

Which triples can be derived from G using rule 9?

Question 3—RDFS Inferencing

Assume the following inference rule:

Rule 9

if $(e, \texttt{rdf:type}, C_1) \wedge (C_1, \texttt{rdfs:subClassOf}, C_2)$
$\rightarrow (e, \texttt{rdf:type}, C_2)$

Let G be a knowledge graph, with triples:

(ex:Victor,	rdf:type,	ex:Teacher)
(ex:Teacher,	rdfs:subClassOf,	ex:EducationalStaff)
(ex:EducationalStaff,	rdfs:subClassOf,	foaf:Person)

Which triples can be derived from G using rule 9?

Answer:

(ex:Victor, rdf:type, ex:EducationalStaff)

Question 3—RDFS Inferencing

Assume the following inference rule:

Rule 9

if $(e, \texttt{rdf:type}, C_1) \wedge (C_1, \texttt{rdfs:subClassOf}, C_2)$
$\rightarrow (e, \texttt{rdf:type}, C_2)$

Let G be a knowledge graph, with triples:

(ex:Victor,	rdf:type,	ex:Teacher)
(ex:Teacher,	rdfs:subClassOf,	ex:EducationalStaff)
(ex:EducationalStaff,	rdfs:subClassOf,	foaf:Person)

Which triples can be derived from G using rule 9?

Answer:

(ex:Victor, rdf:type, ex:EducationalStaff)

If we apply it again, also:

(ex:Victor, rdf:type, foaf:Person)

Question 4—RDFS Inferencing II

Assume the following inference rule:

Rule 11

if $(C_1, \texttt{rdfs:subClassOf}, C_2) \wedge (C_2, \texttt{rdfs:subClassOf}, C_3)$
$\rightarrow (C_1, \texttt{rdfs:subClassOf}, C_3)$

Let G, once again, be a knowledge graph, with triples:

(ex:Victor,	rdf:type,	ex:Teacher)
(ex:Teacher,	rdfs:subClassOf,	ex:EducationalStaff)
(ex:EducationalStaff,	rdfs:subClassOf,	foaf:Person)

Which triples can be derived from G using rule 11?

Question 4—RDFS Inferencing II

Assume the following inference rule:

Rule 11

if $(C_1, \texttt{rdfs:subClassOf}, C_2) \wedge (C_2, \texttt{rdfs:subClassOf}, C_3)$
$\rightarrow (C_1, \texttt{rdfs:subClassOf}, C_3)$

Let G, once again, be a knowledge graph, with triples:

(ex:Victor,	rdf:type,	ex:Teacher)
(ex:Teacher,	rdfs:subClassOf,	ex:EducationalStaff)
(ex:EducationalStaff,	rdfs:subClassOf,	foaf:Person)

Which triples can be derived from G using rule 11?

Answer:

(ex:Teacher, rdfs:subClassOf, ex:Person)

Question 5—RDFS Inferencing III

Assume the following inference rules:

Rule 2

if $(e, p, u) \wedge (p, \texttt{rdfs:domain}, C_1)$
$\rightarrow (e, \texttt{rdf:type}, C_1)$

Rule 3

if $(e, p, v) \wedge (p, \texttt{rdfs:range}, C_2)$
$\rightarrow (v, \texttt{rdf:type}, C_2)$

Let H be a knowledge graph, with triples:

(ex:Victor,	ex:teaches,	ex:KandD)
(ex:teaches,	rdfs:domain,	foaf:Person)
(ex:teaches,	rdfs:range,	ex:Course)

Which triples can be derived from H using rule 2 and 3?

Question 5—RDFS Inferencing III

Assume the following inference rules:

Rule 2

if $(e, p, u) \wedge (p, \texttt{rdfs:domain}, C_1)$
$\rightarrow (e, \texttt{rdf:type}, C_1)$

Rule 3

if $(e, p, v) \wedge (p, \texttt{rdfs:range}, C_2)$
$\rightarrow (v, \texttt{rdf:type}, C_2)$

Let H be a knowledge graph, with triples:

(ex:Victor,	ex:teaches,	ex:KandD)
(ex:teaches,	rdfs:domain,	foaf:Person)
(ex:teaches,	rdfs:range,	ex:Course)

Which triples can be derived from H using rule 2 and 3?
Answer:

(ex:Victor,	rdf:type,	foaf:Person)
(ex:KandD,	rdf:type,	ex:Course)

Question 6—RDFS Inferencing IV

Assume, once more, the following inference rules:

Rule 2

if $(e, p, u) \wedge (p, \texttt{rdfs:domain}, C_1)$
$\rightarrow (e, \texttt{rdf:type}, C_1)$

Rule 3

if $(e, p, v) \wedge (p, \texttt{rdfs:range}, C_2)$
$\rightarrow (v, \texttt{rdf:type}, C_2)$

Task: Think of an intuitive example, other than those already mentioned, for which these rules hold.

Question 6—RDFS Inferencing IV

Possible answer I:

If we have a triples

(ex:VU,	ex:locatedIn,	ex:Amsterdam)
(ex:locatedIn,	rdfs:domain,	ex:University)
(ex:locatedIn,	rdfs:range,	ex:City)

then we can derive (ex:VU, a, ex:University) and (ex:Amsterdam, a, ex:City)

Possible answer II:

If we have a triples

(ex:VanGogh,	ex:painterOf,	ex:TheStarryNight)
(ex:painterOf,	rdfs:domain,	ex:Painter)
(ex:painterOf,	rdfs:range,	ex:Painting)

then we can derive (ex:VanGogh, a, ex:Painter) and (ex:TheStarryNight, a, ex:Painting)

Question 7—RDFS Entailment

Consider, once again, knowledge graph H:

(ex:Victor,	ex:teaches,	ex:KandD)
(ex:teaches,	rdfs:domain,	foaf:Person)
(ex:teaches,	rdfs:range,	ex:Course)

What will happen if we add the following triple to H, and why:

(ex:Victor, ex:teaches, ex:Benno)

Question 7—RDFS Entailment

Consider, once again, knowledge graph H:

(ex:Victor,	ex:teaches,	ex:KandD)
(ex:teaches,	rdfs:domain,	foaf:Person)
(ex:teaches,	rdfs:range,	ex:Course)

What will happen if we add the following triple to H, and why:

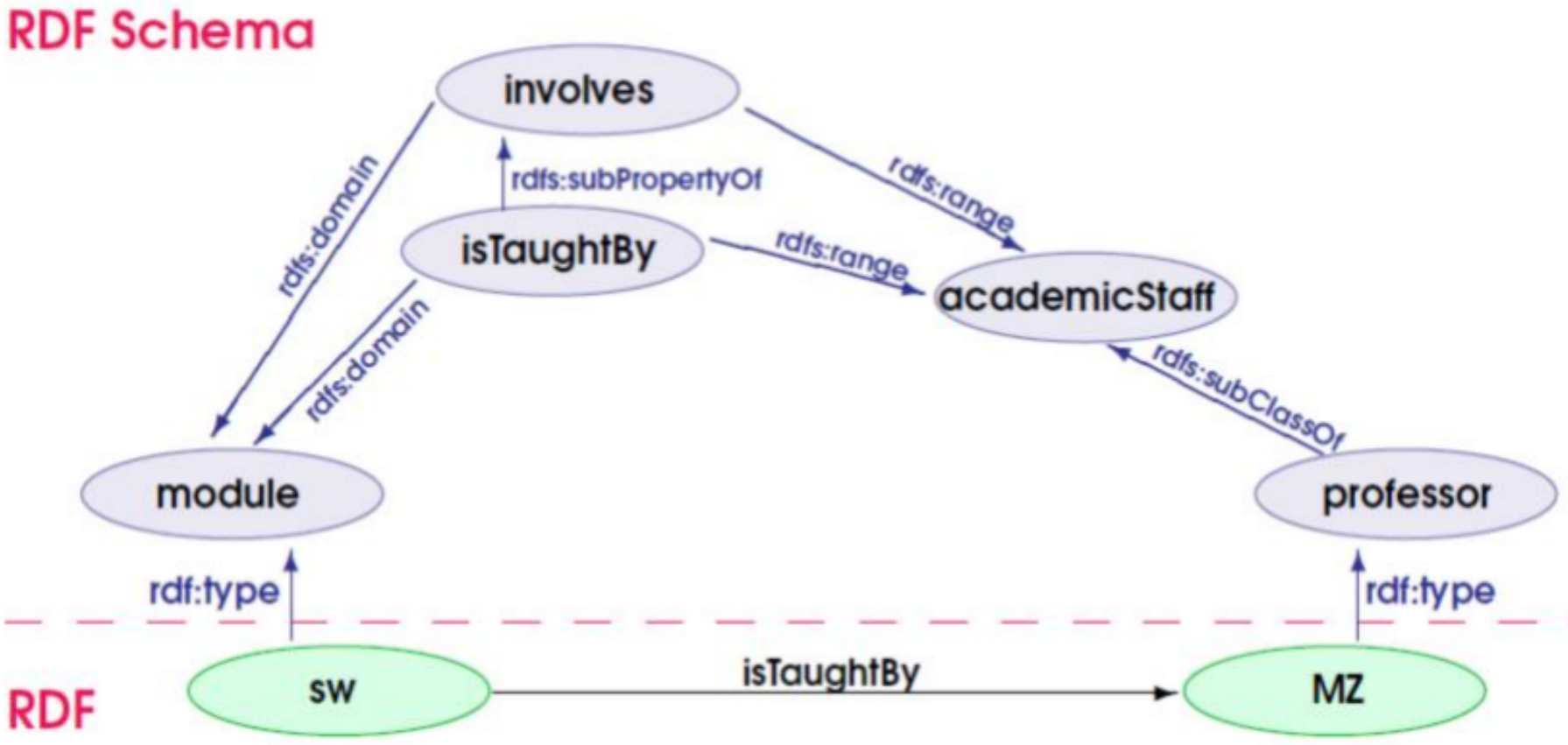
(ex:Victor, ex:teaches, ex:Benno)

Answer:

- Property `ex:teaches` has the class `ex:Course` as range.
- RDFS inferencing now derives that `ex:Benno` is an `ex:Course`.
- To solve this, we need another property with `foaf:Person` as range, and use that to link `ex:Victor` and `ex:Benno` instead.

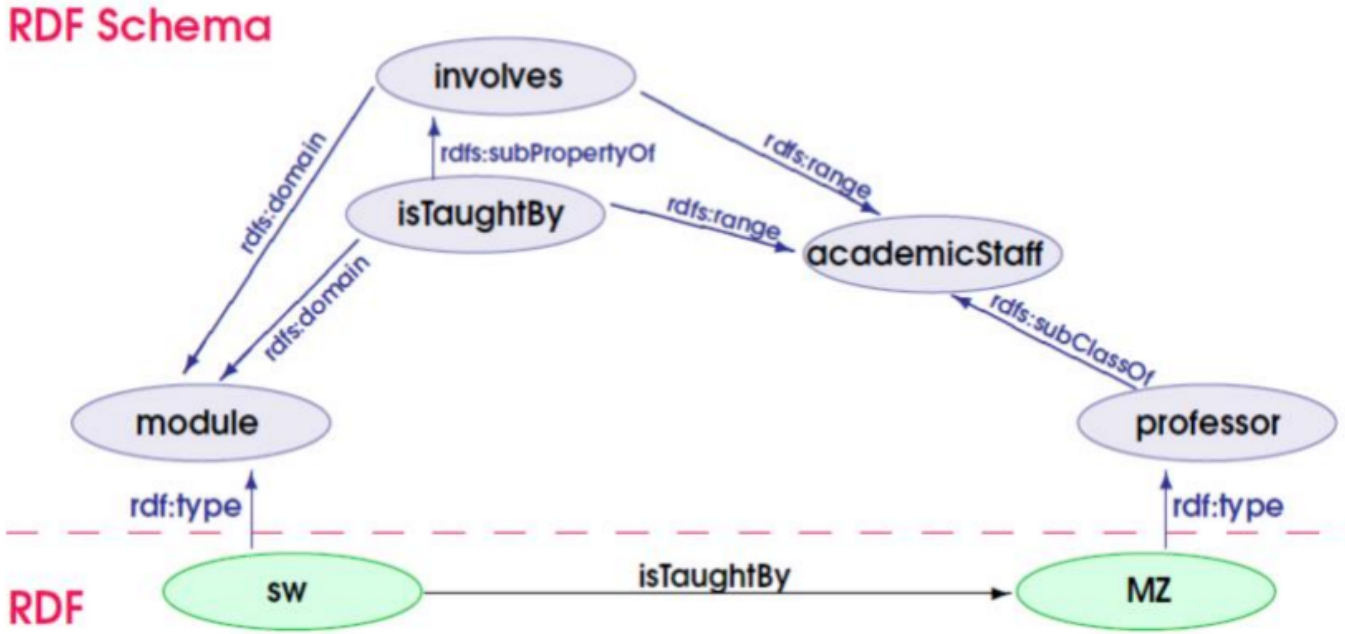
Question 8—RDFS Entailment II

Consider the following knowledge graph:



What other triples can be derived according to RDFS semantics?

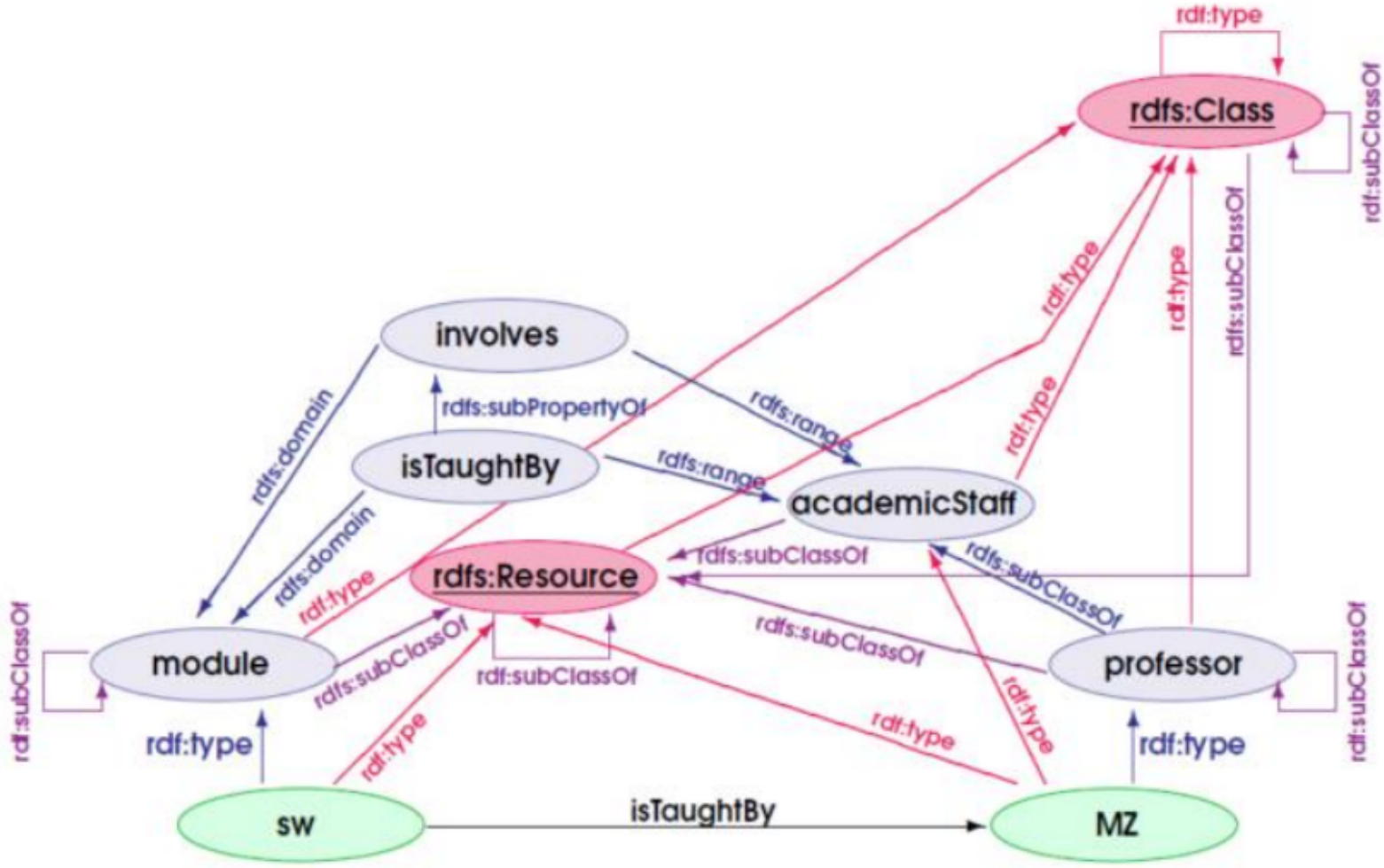
Question 8—RDFS Entailment II



1.	$(s, p, o) \wedge o$ is a literal	$\rightarrow (\_{:}b, \texttt{rdf:type}, \texttt{rdfs:Literal})$
2.	$(s, p, o) \wedge (p, \texttt{rdfs:domain}, C)$	$\rightarrow (s, \texttt{rdf:type}, C)$
3.	$(s, p, o) \wedge (p, \texttt{rdfs:range}, C)$	$\rightarrow (o, \texttt{rdf:type}, C)$
4a.	(s, p, o)	$\rightarrow (s, \texttt{rdf:type}, \texttt{rdfs:Resource})$
4b.	(s, p, o)	$\rightarrow (o, \texttt{rdf:type}, \texttt{rdfs:Resource})$
5.	$(p, \texttt{rdfs:subPropertyOf}, q) \wedge (q, \texttt{rdfs:subPropertyOf}, r)$	$\rightarrow (p, \texttt{rdfs:subPropertyOf}, r)$
6.	$(p, \texttt{rdf:type}, \texttt{rdf:Property})$	$\rightarrow (p, \texttt{rdfs:subPropertyOf}, p)$
7.	$(s, p, o) \wedge (p, \texttt{rdfs:subPropertyOf}, q)$	$\rightarrow (s, q, o)$
8.	$(C, \texttt{rdf:type}, \texttt{rdfs:Class})$	$\rightarrow (C, \texttt{rdfs:subClassOf}, \texttt{rdfs:Resource})$
9.	$(e, \texttt{rdf:type}, C) \wedge (C, \texttt{rdfs:subClassOf}, D)$	$\rightarrow (e, \texttt{rdf:type}, D)$
10.	$(C, \texttt{rdf:type}, \texttt{rdfs:Class})$	$\rightarrow (C, \texttt{rdfs:subClassOf}, C)$
11.	$(C, \texttt{rdfs:subClassOf}, D) \wedge (D, \texttt{rdfs:subClassOf}, E)$	$\rightarrow (C, \texttt{rdfs:subClassOf}, E)$
12.	$(p, \texttt{rdf:type}, \texttt{rdfs:ContainerMembershipProperty})$	$\rightarrow (p, \texttt{rdfs:subPropertyOf}, \texttt{rdfs:member})$
13.	$(l, \texttt{rdf:type}, \texttt{rdfs:Datatype})$	$\rightarrow (l, \texttt{rdfs:subClassOf}, \texttt{rdfs:Literal})$

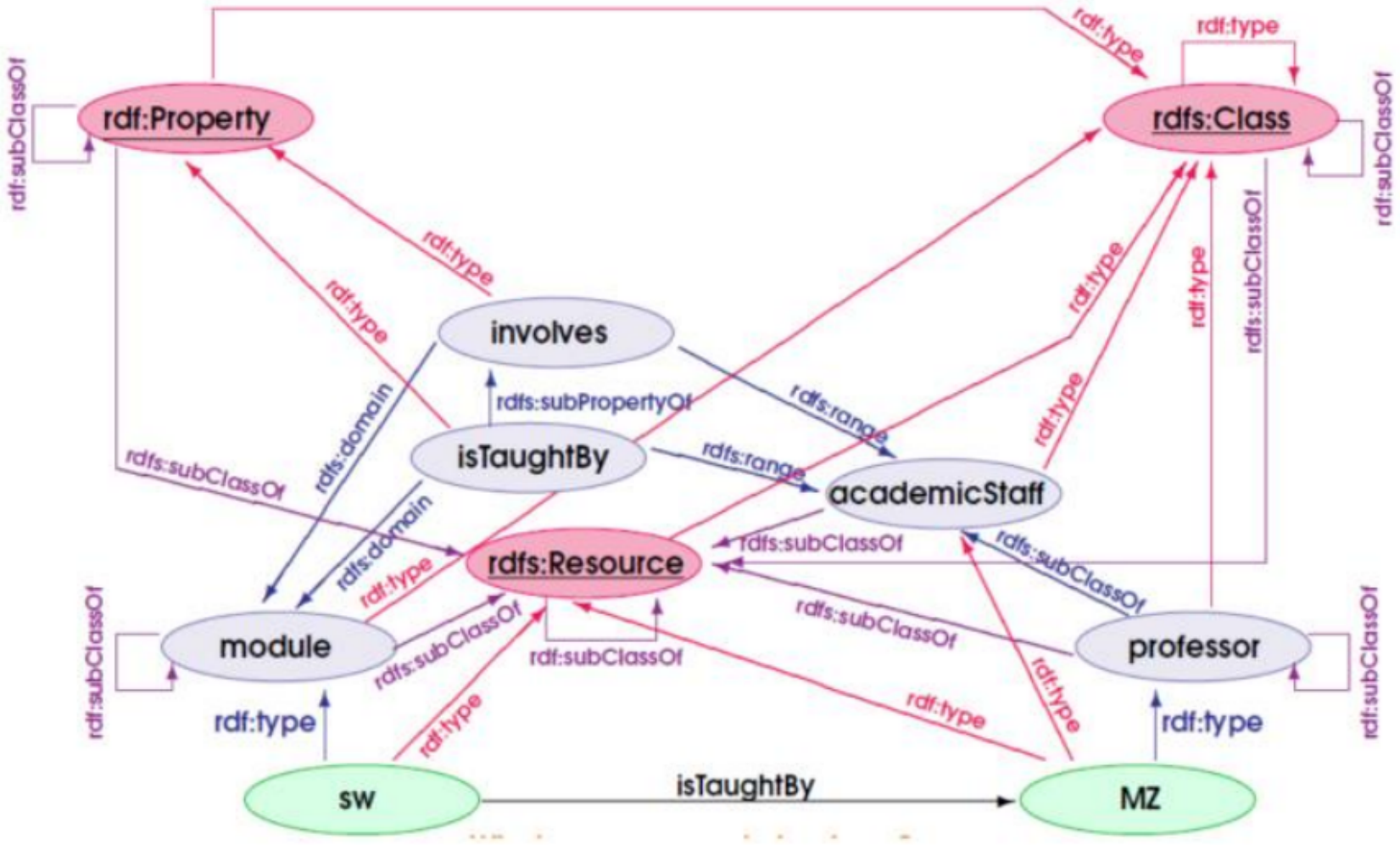
Question 8—RDFS Entailment II

Entailment on types and classes:



Question 8—RDFS Entailment II

Entailment on properties:



Question 9—Modelling with RDFS

Task: Think of a domain and engineer a RDFS knowledge graph, such that it contains *at least*

- 3 instances,
- 3 classes,
- 3 properties, and
- 6 implicit statements.

Question 9—Modelling with RDFS

Example

Domain Astronomy

Scope Inner Solar System

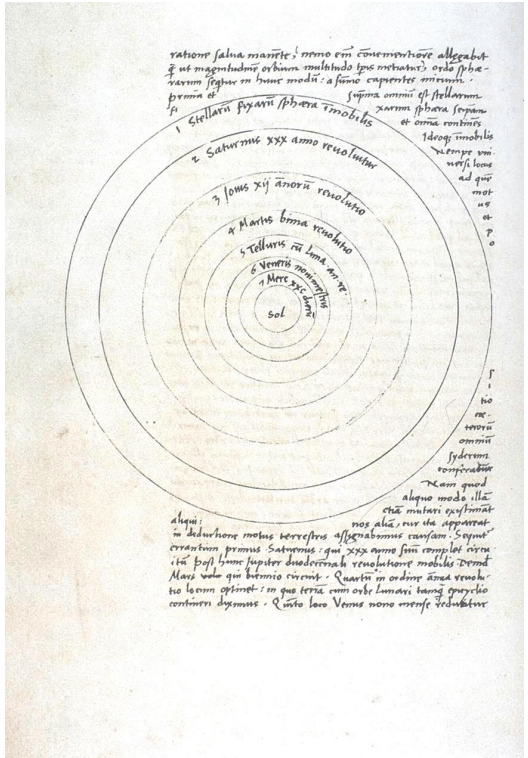


Figure: *De revolutionibus orbium coelestium* (1543), Nicolaus Copernicus

it is not allowed to use this (exact) example in your assignment

Question 9—Modelling with RDFS

Example

Domain Astronomy

Scope Inner Solar System

Classes
- CelestialBody
- Star
- Planet

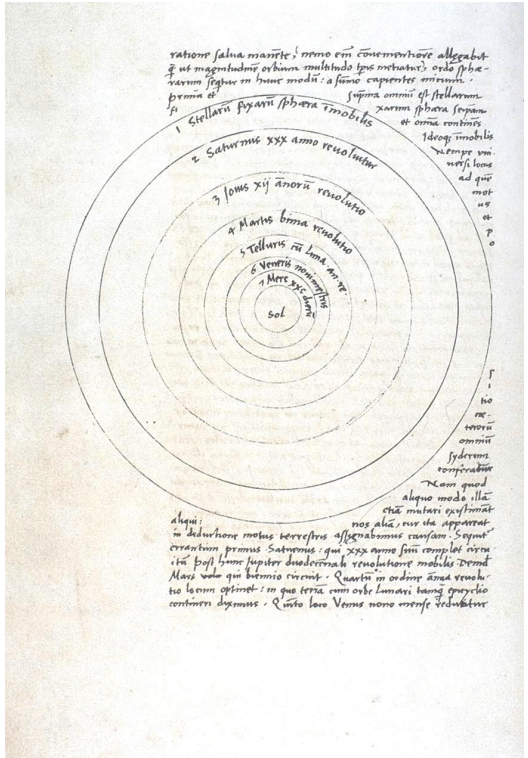


Figure: *De revolutionibus orbium coelestium* (1543), Nicolaus Copernicus

it is not allowed to use this (exact) example in your assignment

Question 9—Modelling with RDFS

Example

Domain Astronomy

Scope Inner Solar System

Classes
- CelestialBody
- Star
- Planet

Properties
- orbits
- population
- humanPopulation

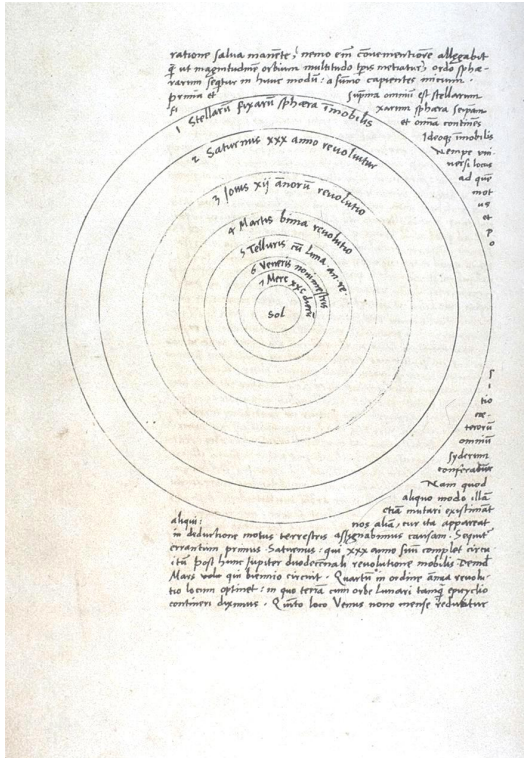


Figure: *De revolutionibus orbium coelestium* (1543), Nicolaus Copernicus

it is not allowed to use this (exact) example in your assignment

Question 9—Modelling with RDFS

Example

Domain Astronomy

Scope Inner Solar System

Classes
- CelestialBody
- Star
- Planet

Properties
- orbits
- population
- humanPopulation

Instances
- Sun
- Earth
- Mars

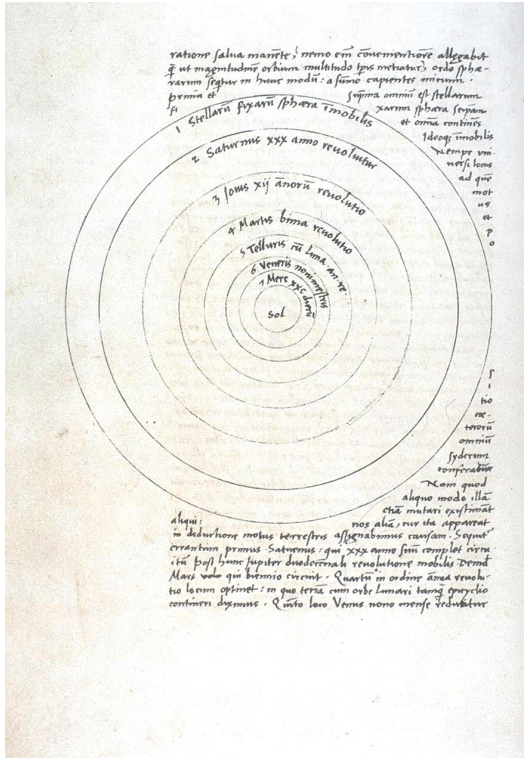


Figure: *De revolutionibus orbium coelestium* (1543), Nicolaus Copernicus

it is not allowed to use this (exact) example in your assignment

Question 9—Modelling with RDFS

Example

Domain Astronomy

Scope Inner Solar System

Classes
- CelestialBody
- Star
- Planet

Properties
- orbits
- population
- humanPopulation

Instances
- Sun
- Earth
- Mars

Implicit statements
- subClassOf
- subPropertyOf

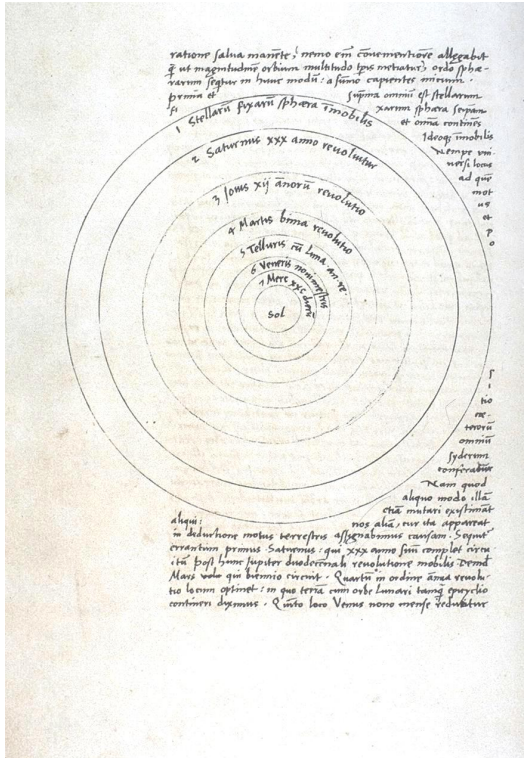


Figure: *De revolutionibus orbium coelestium* (1543), Nicolaus Copernicus

it is not allowed to use this (exact) example in your assignment

Question 9—Modelling with RDFS

```
ex:   "http://example.com/kad/astronomy/"
```

Classes

(ex:CelestialBody,	rdf:type,	rdfs:Class)
(ex:CelestialBody,	rdfs:label,	"Celestial Body"@en)

Question 9—Modelling with RDFS

```
ex:  "http://example.com/kad/astronomy/"
```

Classes

(ex:CelestialBody,	rdf:type,	rdfs:Class)
(ex:CelestialBody,	rdfs:label,	"Celestial Body"@en)
(ex:Star,	rdf:type,	rdfs:Class)
(ex:Star,	rdfs:subClassOf,	ex:CelestialBody)
(ex:Star,	rdfs:label,	"Star"@en)

Question 9—Modelling with RDFS

```
ex:  "http://example.com/kad/astronomy/"
```

Classes

(ex:CelestialBody,	rdf:type,	rdfs:Class)
(ex:CelestialBody,	rdfs:label,	"Celestial Body"@en)
(ex:Star,	rdf:type,	rdfs:Class)
(ex:Star,	rdfs:subClassOf,	ex:CelestialBody)
(ex:Star,	rdfs:label,	"Star"@en)
(ex:Planet,	rdf:type,	rdfs:Class)
(ex:Planet,	rdfs:subClassOf,	ex:CelestialBody)
(ex:Planet,	rdfs:label,	"Planet"@en)
(ex:Satellite,	rdf:type,	rdfs:Class)
(ex:Satellite,	rdfs:subClassOf,	ex:CelestialBody)
(ex:Satellite,	rdfs:label,	"Natural Satellite"@en)

Question 9—Modelling with RDFS

Properties

(ex:population,	rdf:type,	rdf:Property)
(ex:population,	rdfs:label,	"Population count"@en)
(ex:population,	rdfs:domain,	ex:CelestialBody)
(ex:population,	rdfs:range,	xsd:nonNegativeInteger)

Question 9—Modelling with RDFS

Properties

(ex:population,	rdf:type,	rdf:Property)
(ex:population,	rdfs:label,	"Population count"@en)
(ex:population,	rdfs:domain,	ex:CelestialBody)
(ex:population,	rdfs:range,	xsd:nonNegativeInteger)
(ex:humanPopulation,	rdf:type,	rdf:Property)
(ex:humanPopulation,	rdfs:subPropertyOf,	ex:population)
(ex:humanPopulation,	rdfs:label,	"Human population count"@en)
(ex:humanPopulation,	rdfs:domain,	ex:CelestialBody)
(ex:humanPopulation,	rdfs:range,	xsd:nonNegativeInteger)

Question 9—Modelling with RDFS

Properties

(ex:population,	rdf:type,	rdf:Property)
(ex:population,	rdfs:label,	"Population count"@en)
(ex:population,	rdfs:domain,	ex:CelestialBody)
(ex:population,	rdfs:range,	xsd:nonNegativeInteger)
(ex:humanPopulation,	rdf:type,	rdf:Property)
(ex:humanPopulation,	rdfs:subPropertyOf,	ex:population)
(ex:humanPopulation,	rdfs:label,	"Human population count"@en)
(ex:humanPopulation,	rdfs:domain,	ex:CelestialBody)
(ex:humanPopulation,	rdfs:range,	xsd:nonNegativeInteger)
(ex:robotPopulation,	rdf:type,	rdf:Property)
(ex:robotPopulation,	rdfs:subPropertyOf,	ex:population)
(ex:robotPopulation,	rdfs:label,	"Robot population count"@en)
(ex:robotPopulation,	rdfs:domain,	ex:CelestialBody)
(ex:robotPopulation,	rdfs:range,	xsd:nonNegativeInteger)
(ex:orbits,	rdf:type,	rdf:Property)
(ex:orbits,	rdfs:label,	"Orbits"@en)
(ex:orbits,	rdfs:domain,	ex:CelestialBody)
(ex:orbits,	rdfs:range,	ex:CelestialBody)

Question 9—Modelling with RDFS

Instances

(ex:Sun,	rdf:type,	ex:Star)
(ex:Sun,	rdfs:label,	"Sun"@en)

Question 9—Modelling with RDFS

Instances

(ex:Sun,	rdf:type,	ex:Star)
(ex:Sun,	rdfs:label,	"Sun"@en)
(ex:Earth,	rdf:type,	ex:Planet)
(ex:Earth,	rdfs:label,	"Earth"@en)
(ex:Earth,	ex:humanPopulation,	"7900000000"^^xsd:nonNegativeInteger)
(ex:Earth,	ex:orbits,	ex:Sun)

Question 9—Modelling with RDFS

Instances

(ex:Sun,	rdf:type,	ex:Star)
(ex:Sun,	rdfs:label,	"Sun"@en)
(ex:Earth,	rdf:type,	ex:Planet)
(ex:Earth,	rdfs:label,	"Earth"@en)
(ex:Earth,	ex:humanPopulation,	"7900000000"^^xsd:nonNegativeInteger)
(ex:Earth,	ex:orbits,	ex:Sun)
(ex:Moon,	rdf:type,	ex:Satellite)
(ex:Moon,	rdfs:label,	"Earth's moon"@en)
(ex:Moon,	ex:population,	"0"^^xsd:nonNegativeInteger)
(ex:Moon,	ex:orbits,	ex:Earth)
(ex:Mars,	rdf:type,	ex:Planet)
(ex:Mars,	rdfs:label,	"Mars"@en)
(ex:Mars,	ex:robotPopulation,	"6"^^xsd:nonNegativeInteger)
(ex:Mars,	ex:orbits,	ex:Sun)

Question 9—Modelling with RDFS

Implicit knowledge?

(ex:Sun,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Earth,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Moon,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Mars,	rdf:type,	ex:CelestialBody)	rule 9

Question 9—Modelling with RDFS

Implicit knowledge?

(ex:Sun,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Earth,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Moon,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Mars,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Mars,	ex:population,	"6"^^xsd:nonNegativeInteger)	rule 7

Question 9—Modelling with RDFS

Implicit knowledge?

(ex:Sun,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Earth,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Moon,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Mars,	rdf:type,	ex:CelestialBody)	rule 9
(ex:Mars,	ex:population,	"6"^^xsd:nonNegativeInteger)	rule 7
(ex:Sun,	rdf:type,	rdfs:Resource)	rule 4
(ex:CelestialBody,	rdf:type,	rdfs:Resource)	rule 4
(ex:CelestialBody,	rdfs:subClassOf,	rdfs:Resource)	rule 8
(ex:CelestialBody,	rdfs:subClassOf,	ex:CelestialBody)	rule 10
...			

plus many more

Question 10—Shortcomings of RDFS

Consider the following knowledge graph K:

(ex:speaksWith,	rdfs:domain,	ex:HomoSapien)
(ex:HomoSapien,	rdfs:subClassOf,	ex:Primate)

Explain

1. why would we expect K to entail statement

   (ex:speaksWith, rdfs:domain, ex:Primate)

2. Explain why this statement is not derivable via RDFS semantics.

Question 10—Shortcomings of RDFS

Consider the following knowledge graph K:

(ex:speaksWith,	rdfs:domain,	ex:HomoSapien)
(ex:HomoSapien,	rdfs:subClassOf,	ex:Primate)

Explain

1. why would we expect K to entail statement

   (ex:speaksWith, rdfs:domain, ex:Primate)

   From axiom 2 we know every instance of `ex:HomoSapien` is also a `ex:Primate`, and axiom 1 tells us that the subject in a `ex:speaksWith` relation is an `ex:HomoSapien`, which suggests that the domain of `ex:speaksWith` is also `ex:Primate`.

2. Explain why this statement is not derivable via RDFS semantics.

Question 10—Shortcomings of RDFS

Consider the following knowledge graph K:

(ex:speaksWith, rdfs:domain, ex:HomoSapien)
(ex:HomoSapien, rdfs:subClassOf, ex:Primate)

Explain

1. why would we expect K to entail statement

   (ex:speaksWith, rdfs:domain, ex:Primate)

   From axiom 2 we know every instance of `ex:HomoSapien` is also a `ex:Primate`, and axiom 1 tells us that the subject in a `ex:speaksWith` relation is an `ex:HomoSapien`, which suggests that the domain of `ex:speaksWith` is also `ex:Primate`.

2. Explain why this statement is not derivable via RDFS semantics.

   The class `ex:HomoSapien` is the same or more specific than `ex:Primate`, its superclass (axiom 2). Without more information we thus cannot exclude the possibility that there are (classes of) primates who *cannot* speak.

Question 11—Statements

Consider the following three statements:

1. (ex:e, foaf:knows, ex:u)
2. (ex:e, rdf:type, ex:u)
3. (ex:e, p, ex:u)

Explain the conceptual difference between these three statements.

Question 11—Statements

Consider the following three statements:

1. (ex:e, foaf:knows, ex:u)
2. (ex:e, rdf:type, ex:u)
3. (ex:e, p, ex:u)

Explain the conceptual difference between these three statements.

Answer:

- Statement 1 has *social semantics*. Since it is common knowledge that the FOAF vocabulary is used to connect people, we implicitly expect `ex:e` and `ex:u` to be of that type. However, this is not formally specified (e.g. (`ex:e`, `rdf:type`, `foaf:Person`)), and cannot be inferred.

Question 11—Statements

Consider the following three statements:

1. (ex:e, foaf:knows, ex:u)
2. (ex:e, rdf:type, ex:u)
3. (ex:e, p, ex:u)

Explain the conceptual difference between these three statements.

Answer:

- Statement 1 has *social semantics*. Since it is common knowledge that the FOAF vocabulary is used to connect people, we implicitly expect `ex:e` and `ex:u` to be of that type. However, this is not formally specified (e.g. (`ex:e`, `rdf:type`, `foaf:Person`)), and cannot be inferred.
- Statement 2 has *formal semantics*: if, for example, we know that (`ex:u`, `rdfs:subClassOf`, `ex:v`), then we can infer that (`ex:e`, `rdf:type`, `ex:v`).

Question 11—Statements

Consider the following three statements:

1. (ex:e, foaf:knows, ex:u)
2. (ex:e, rdf:type, ex:u)
3. (ex:e, p, ex:u)

Explain the conceptual difference between these three statements.

Answer:

- Statement 1 has *social semantics*. Since it is common knowledge that the FOAF vocabulary is used to connect people, we implicitly expect `ex:e` and `ex:u` to be of that type. However, this is not formally specified (e.g. (`ex:e`, `rdf:type`, `foaf:Person`)), and cannot be inferred.
- Statement 2 has *formal semantics*: if, for example, we know that (`ex:u`, `rdfs:subClassOf`, `ex:v`), then we can infer that (`ex:e`, `rdf:type`, `ex:v`).
- Statement 3 has neither social or formal semantics associated to it.

Quiz

To test your proficiency:

Exercise quiz Module 3

(see the course's Canvas page)

NB: completing the quiz is mandatory to continue to next week's module